# Recurring costs — quiet monthly drains made visible

**Epic:** EPIC-015 · **Requirements:** A-17, A-22, A-03 · **Schema:** §3.6 recurring, §5 document ids, §6.4 writes

> **Status:** the data layer with atomic confirm and skip is done (#197). The dashboard "Due" card (#200) and the Recurring screen (#203) are in progress. Their owner adds those parts and the screenshots when they merge.

## Problem

Kavindu's spending was fragmented. A gym auto-debit and two subscriptions he meant to cancel kept draining money he never saw (**A-17**). Utilities make his effective rent vary every month, so a fixed amount is wrong (**A-22**). After a gap in entries, he needs to catch up quickly, without the app showing failure (**A-03**).

## Design decision

- **Templates, not generated entries.** A recurring template stores the name, category, account and expected amount, whether the amount varies, the day of month (1–28 only, so nothing needs clamping), and the stored cursor `nextDueDate`.
- **A due item is derived, never stored.** It is due when the template isn't paused and `nextDueDate ≤ today`. Nothing is generated in the background, so nothing can be generated twice (#200).
- **Confirm and skip move the cursor** by one month:
  - **Confirm** is one `WriteBatch`: the `ACTUAL` `EXPENSE` transaction (with `recurringId`) and the template's new `nextDueDate`, or neither.
  - **Skip** is a single `update` of the cursor.
- **No double confirms.** `RecurringRepositoryImpl` confirms or skips only a due, unpaused template, under a mutex.
  - It remembers the cursor it just advanced, because the new `nextDueDate` reaches Room only when the sync listener fires, and a fast second tap would still see the old one.
  - The confirmed transaction's id is `{recurringId}_{yyyyMM}` of the due month (schema §5). A confirm from a second device for the same month targets the same document, and the rules (`keepsCreatedAt`) reject the second create.
- **Fixed vs variable** (A-22): a fixed template confirms only for its expected amount, in one tap. A variable one (utilities) opens an amount sheet prefilled with the expected amount, and stores both amounts (`expectedAmountLkrMinor` on the transaction).
- **A recurring cost is committed by definition**, so confirmed entries are saved as `COMMITTED` whatever the category's default.
- **Removing a template** is a soft delete (`deletedAt`); the sync mirror drops it from Room. Hard deletes are denied by the rules.

## Data flow

```
Dashboard "Due" card — DashboardViewModel — GetDueRecurringUseCase (#200) ── RecurringRepository.observeAll ── Room recurring
      Confirm / Skip — confirm / skip use cases (#200) ── RecurringRepository.confirm / skip
                                                                                  └─ Firestore WriteBatch (transaction + template cursor)
Recurring screen ─── RecurringViewModel (#203) ── list + monthly committed total ── RecurringRepository ── Room
Sync — SyncManager listener on users/{uid}/recurring ── Room (soft-deleted templates removed)
```

## Edge cases handled

| Case | Behaviour |
|---|---|
| Two quick taps on Confirm | The second is refused: the cursor already moved, even before Room catches up. `RecurringRepositoryImplTest` covers sequential and concurrent confirms. |
| Two months missed after a gap (A-03) | Each missed month is due in turn. Confirming moves to the next one, until the template is no longer due. |
| Confirming before the due date | Refused ("not due until …"). |
| A paused template | Never due; confirm and skip are refused. |
| A fixed amount confirmed with a different value | Refused. Only variable templates accept another amount. |
| Editing a template from a stale copy | The update map leaves out `nextDueDate`, so an edit can't rewind the cursor and make a paid month due again. |
| The template is missing on the server | `update()` in the batch fails it, so no orphan expense is written. |
| Two devices offline confirm the same month | The same transaction id, so the second create is rejected by the rules. Schema §3.6 records the stored cursor's drift risk. |

## Screenshots

| **TODO(M4) img/recurring-due.png — Recurring costs on seeded data** |
|---|
| Seeded data: utilities due this month, and rent, gym and two subscriptions confirmed. Captured once #200 or #203 merges. Until then the figure is the S6 wireframe (S6-insights-recurring.md §SCR-50). |

## Tests

- `RecurringRepositoryImplTest`: confirm, double and concurrent confirm, overdue catch-up, not due, paused, fixed vs variable, skip, signed out.
- `DtoMappingTest` `Recurring*`: round-trip, the schema field names, the update map leaving out the cursor, and calendar-day stability across time zones.
- `RoomMappersTest`: the recurring round-trip.
- `SyncManagerTest`: the recurring mirror and soft-delete removal.